
Base Information

Name: Sergio Ley Languren

Language Used: Python

Platform: MacOS

CPU: Apple M1 Pro

RAM: 16gb

Complexity Analysis

Time Complexity Derivation

The brute force ThreeSum algorithm works by checking every possible triplet of distinct indices in the given array and determining whether the sum of those three values equals the target. To understand the time complexity, we first count how many triplets the algorithm actually examines. The indices are chosen such that $i < j < k$, which prevents repeated triplets. We can use and derive the binomial coefficient equation: $C(n, 3) = \frac{n(n-1)(n-2)}{6}$. $(n-1)$ and $(n-2)$ are derived from the limitation within the algorithm in which each inner loop cannot pick the same element as one of their parent loops. If we factor the nominator of the fraction we would get: $n^3 - 3n^2 + 2n$. We will keep in mind for the following that the denominator “6” was found by keeping in mind that $3! = 3 \cdot 2 \cdot 1 = 6$ which makes the equation $\frac{1}{6}n^3 - \frac{3}{6}n^2 + \frac{2}{6}n$. The derivation ultimately simplifies too: $O(n^3)$. We must keep in mind that Big-O notation will only care about the fastest growing constant and will ignore all other constants, hence, we get $O(n^3)$.

Space Complexity Analysis

The space complexity is much simpler to analyze. Aside from the input array, which takes $O(n)$ space, the algorithm only uses a fixed number of scalar variables: loop indices, the result variable, and the operation counter. There are no additional data structures whose size depends on n . For this reason, the auxiliary space complexity is $O(1)$, or constant space. This means the algorithm’s memory usage remains stable regardless of input size.

Best, Average, and Worst Case

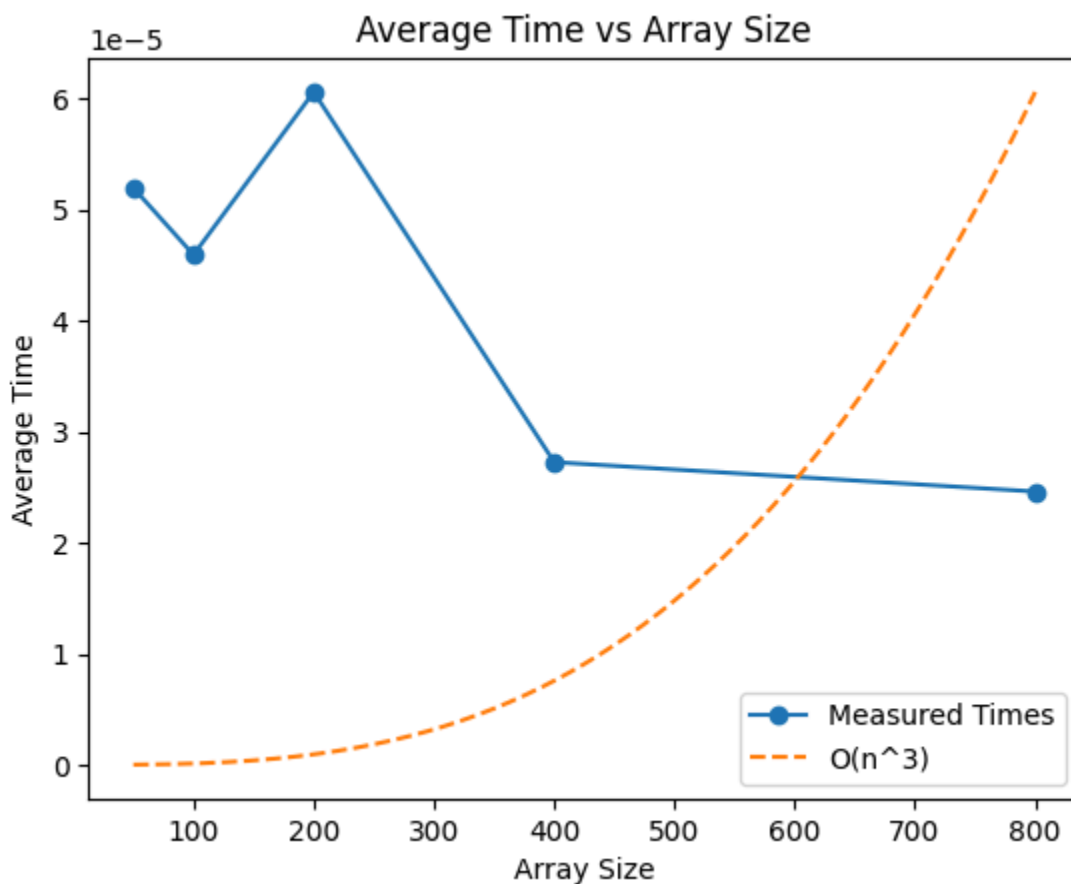
When analyzing best, average, and worst cases, it is important to distinguish between theoretical growth and practical behavior. In the best case, a valid triplet is discovered almost immediately, perhaps in the very first iteration. In this scenario, the algorithm halts early, and only a constant

number of operations are performed, resulting in a best-case runtime of $\Omega(1)$. The average case occurs when the desired triplet is found somewhere in the middle of the search. On average, the algorithm will examine about half of all possible triplets before stopping; however, this still corresponds to cubic growth, so the average case remains * The worst case happens when no valid triplet exists, or when the only valid triplet is discovered in the very last iteration. In these

cases, the algorithm must exhaustively examine all $\frac{n(n-1)(n-2)}{6}$ triplets. The worst case is the most important in an analysis as it will help us determine how long an algorithm will take in the least-favorable situation, especially when dealing with large or variable inputs.

Results

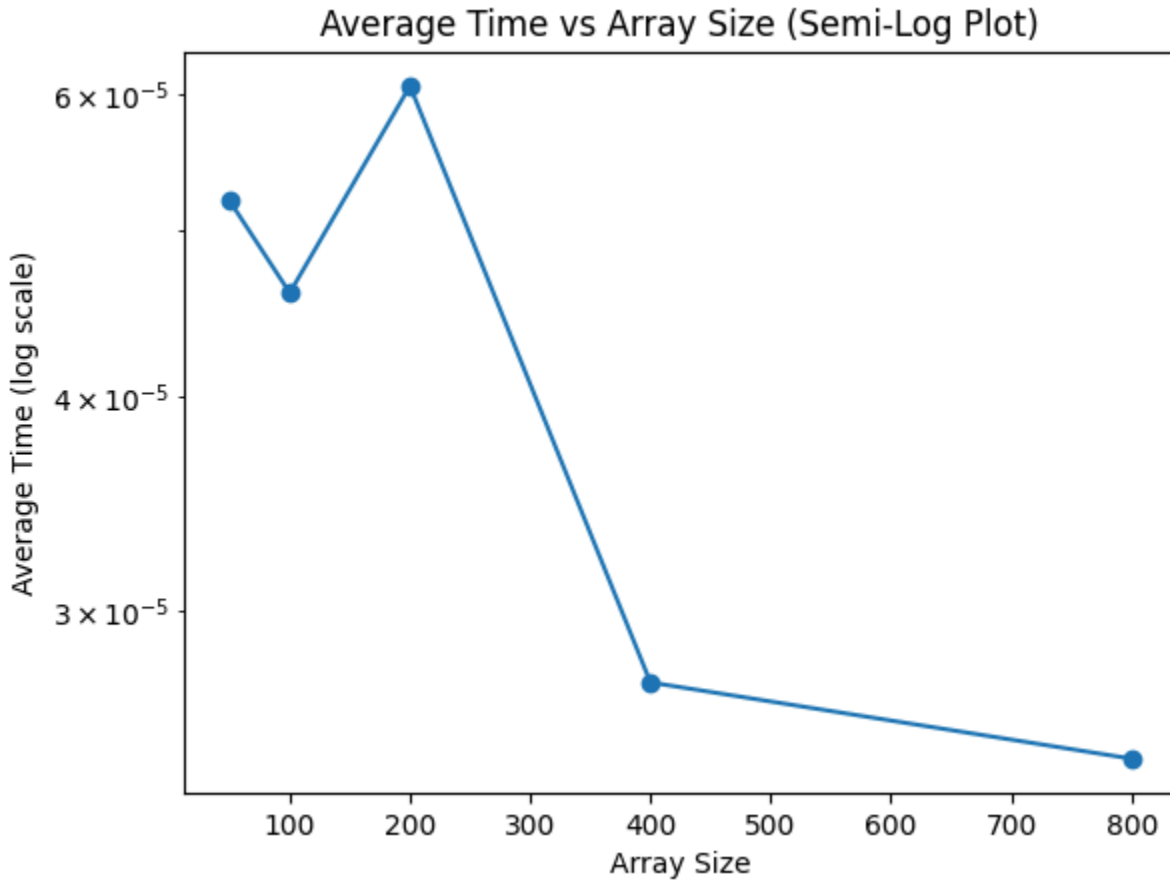
- Figure 1.1



- Figure 1.2



- Figure 1.3



- Table 1.1

	# Array Sizes	# Runtimes	# Operation Count
0	50	5.195010453462601e-05	419.0
1	100	4.592480836436152e-05	345.0
2	200	6.058760336600244e-05	509.0
3	400	2.7254113228991626e-05	84.0
4	800	2.460820251144469e-05	37.0

Comparative Analysis

One efficient approach to the **ThreeSum** problem is the **sorting and two-pointer method**. The array is first sorted, allowing duplicates to be easily skipped. For each element, two pointers are used—one starting just after the current element and one at the end of the array—to find pairs that sum to the negative of the current element. When the sum of the three numbers is zero, the triplet is stored, and the pointers are adjusted to skip duplicates. This reduces the time complexity from $O(n^3)$ to $O(n^2)$.

Another efficient approach is to use a **hash-based method**. For each element in the array, the algorithm uses a hash set to check if a complementary pair (that makes the total sum zero) exists within the remaining elements. This allows constant-time lookups for the needed values while iterating through the list. Though it still averages $O(n^2)$ time complexity, it can be faster in practice for unsorted input since no sorting is required. However, it requires $O(n)$ extra space for the hash set, trading memory for speed.